

USE CASES DELIVERABLE #2

Decorator Pattern (Announcements and Communication)

// 1. The Component Interface (Abstract Product)

// Defines the interface for objects that can have responsibilities

// added to them dynamically.

interface Notification {

void send(String message);

}

// 2. The Concrete Component (The "Real" Object)

// Defines an object to which additional responsibilities can be attached.

// In your project, this is the basic internal dashboard post.

class DashboardAnnouncement implements Notification {

@Override

public void send(String message) {

System.out.println("LOG: Announcement posted to School Dashboard: " + message);

}

}

// 3. The Decorator (The Wrapper)

// Maintains a reference to a Component object and defines an interface

// that conforms to Component's interface.

abstract class NotificationDecorator implements Notification {

protected Notification tempNotification; // Reference to the wrapped object

```

public NotificationDecorator(Notification notification) {
    this.tempNotification = notification;
}

@Override
public void send(String message) {
    tempNotification.send(message);
}
}

// 4. Concrete Decorator A
// Adds responsibilities to the component (Email functionality).
class EmailDecorator extends NotificationDecorator {
    public EmailDecorator(Notification notification) {
        super(notification);
    }

    @Override
    public void send(String message) {
        super.send(message); // Execute the original behavior
        sendEmail(message); // Add the new behavior
    }

    private void sendEmail(String message) {
        System.out.println("ACTION: Email alert sent to all subscribed Students/Teachers.");
    }
}

```

```
}  
}
```

```
// 5. Concrete Decorator B
```

```
// Adds responsibilities to the component (SMS functionality).
```

```
class SMSDecorator extends NotificationDecorator {
```

```
    public SMSDecorator(Notification notification) {  
        super(notification);  
    }
```

```
@Override
```

```
public void send(String message) {  
    super.send(message); // Execute the original behavior  
    sendSMS(message);    // Add the new behavior  
}
```

```
private void sendSMS(String message) {  
    System.out.println("ACTION: SMS emergency alert sent to Parent contacts.");  
}
```

```
}
```

```
// 6. Client Code (The "Shape" of your Application)
```

```
// Demonstrates how to use the pattern in your School Management System.
```

```
public class CommunicationModule {
```

```
    public static void main(String[] args) {
```

```
System.out.println("--- Scenario 1: Basic Announcement ---");
```

```
Notification simpleNote = new DashboardAnnouncement();
```

```
simpleNote.send("The library will close at 5 PM today.");
```

```
System.out.println("\n--- Scenario 2: Important Announcement (Dashboard + Email) ---");
```

```
Notification importantNote = new EmailDecorator(new DashboardAnnouncement());
```

```
importantNote.send("Exam schedules have been published.");
```

```
System.out.println("\n--- Scenario 3: URGENT Emergency (Dashboard + Email + SMS) ---");
```

```
Notification urgentNote = new SMSDecorator(  
    new EmailDecorator(  
        new DashboardAnnouncement()));
```

```
urgentNote.send("School is closed tomorrow due to heavy rain.");
```

```
}
```

```
}
```

```
-----  
----
```

```
*Decorator Pattern (Generate Reports)*
```

```
-----
```

```
// 1. Component Interface
```

```
interface Report {
```

```
    String getHeader();
```

```
    String getContent();
```

```
}
```

// 2. Concrete Component

```
class StudentTranscript implements Report {  
    @Override  
    public String getHeader() {  
        return "OFFICIAL STUDENT TRANSCRIPT\nSchool: Global Academy\n";  
    }  
  
    @Override  
    public String getContent() {  
        return "Student: Yassin Sameh\nGPA: 3.8\nStatus: Passed";  
    }  
}
```

// 3. Base Decorator

```
abstract class ReportDecorator implements Report {  
    protected Report tempReport;  
  
    public ReportDecorator(Report report) {  
        this.tempReport = report;  
    }  
  
    @Override  
    public String getHeader() {  
        return tempReport.getHeader();  
    }  
}
```

```
@Override
public String getContent() {
    return tempReport.getContent();
}
}
```

// 4. Concrete Decorator A: Adds a Watermark

```
class WatermarkDecorator extends ReportDecorator {
    public WatermarkDecorator(Report report) {
        super(report);
    }
}
```

```
@Override
public String getContent() {
    return super.getContent() + "\n[WATERMARK: CONFIDENTIAL DOCUMENT]";
}
}
```

// 5. Concrete Decorator B: Adds a Digital Signature

```
class DigitalSignatureDecorator extends ReportDecorator {
    public DigitalSignatureDecorator(Report report) {
        super(report);
    }
}
```

```
@Override
public String getHeader() {
```

```

        return super.getHeader() + "[SIGNED BY PRINCIPAL: Verified ✓]\n";
    }
}

```

// 6. Client Code

```

public class ReportModule {

    public static void main(String[] args) {

        // Basic Report

        Report basicTranscript = new StudentTranscript();

        Report officialTranscript = new DigitalSignatureDecorator(
            new WatermarkDecorator(
                new StudentTranscript()));

        System.out.println("--- PRINTING OFFICIAL TRANSCRIPT ---");
        System.out.println(officialTranscript.getHeader());
        System.out.println(officialTranscript.getContent());
    }
}

```

Use Case 1 with bridge pattern: Assignment Submission System:// Implementor

```

interface Storage {

    void store(String fileName);

}

```

// Concrete Implementors

```
class LocalStorage implements Storage {  
    public void store(String fileName) {  
        System.out.println("Storing " + fileName + " locally");  
    }  
}
```

```
class CloudStorage implements Storage {  
    public void store(String fileName) {  
        System.out.println("Storing " + fileName + " in cloud");  
    }  
}
```

// Abstraction

```
abstract class Assignment {  
    protected Storage storage;  
  
    public Assignment(Storage storage) {  
        this.storage = storage;  
    }  
  
    abstract void submit(String fileName);  
}
```

// Refined Abstraction

```
class Homework extends Assignment {  
    public Homework(Storage storage) {
```

```

        super(storage);
    }

    public void submit(String fileName) {
        System.out.println("Submitting homework:");
        storage.store(fileName);
    }
}

```

```

class Project extends Assignment {
    public Project(Storage storage) {
        super(storage);
    }

    public void submit(String fileName) {
        System.out.println("Submitting project:");
        storage.store(fileName);
    }
}

```

Use Case 2 by bridge pattern: Exam Notification System :// Implementor

```

interface NotificationSender {
    void sendMessage(String message);
}

```

```

class EmailSender implements NotificationSender {

```

```
    public void sendMessage(String message) {  
        System.out.println("Email: " + message);  
    }  
}
```

```
class SMSSender implements NotificationSender {  
    public void sendMessage(String message) {  
        System.out.println("SMS: " + message);  
    }  
}
```

// Abstraction

```
abstract class ExamNotification {  
    protected NotificationSender sender;  
  
    public ExamNotification(NotificationSender sender) {  
        this.sender = sender;  
    }  
  
    abstract void notifyStudent();  
}
```

// Refined

```
class MidtermNotification extends ExamNotification {  
    public MidtermNotification(NotificationSender sender) {  
        super(sender);  
    }  
}
```

```

    }

    public void notifyStudent() {
        sender.sendMessage("Midterm exam scheduled");
    }
}

```

```

class FinalNotification extends ExamNotification {
    public FinalNotification(NotificationSender sender) {
        super(sender);
    }

    public void notifyStudent() {
        sender.sendMessage("Final exam scheduled");
    }
}

```

Use Case3 by flyweight pattern : Classroom Seat Management : // Flyweight

```

class SeatType {
    private String type;

    public SeatType(String type) {
        this.type = type;
    }

    public void display(int seatNumber) {

```

```

        System.out.println("Seat " + seatNumber + " is " + type);
    }
}

// Flyweight Factory
import java.util.HashMap;

class SeatFactory {
    private static HashMap<String, SeatType> seats = new HashMap<>();

    public static SeatType getSeatType(String type) {
        if (!seats.containsKey(type)) {
            seats.put(type, new SeatType(type));
        }
        return seats.get(type);
    }
}

```

//use case 1

//Legacy Grade Export

// Target Interface

```

public interface GradeExporter {
    void exportGrades(String studentId, String reportData);
}

```

// Adaptee: existing legacy class with incompatible interface

```
public class LegacyReportPrinter {  
    public void printReport(String data) {  
        System.out.println("[Legacy Printer] Printing report: " + data);  
    }  
}
```

// Adapter

```
public class GradeExporterAdapter implements GradeExporter {
```

```
    private LegacyReportPrinter adaptee;
```

```
    public GradeExporterAdapter(LegacyReportPrinter adaptee) {
```

```
        this.adaptee = adaptee;
```

```
    }
```

```
@Override
```

```
public void exportGrades(String studentId, String reportData) {
```

```
    String formattedData = "Student ID: " + studentId + " | " + reportData;
```

```
    adaptee.printReport(formattedData);
```

```
}
```

```
}
```

// Client

```
public class ReportService {
```

```
    private GradeExporter exporter;
```

```

public ReportService(GradeExporter exporter) {
    this.exporter = exporter;
}

public void generateReport(String studentId, String data) {
    exporter.exportGrades(studentId, data);
}

public static void main(String[] args) {
    LegacyReportPrinter legacy = new LegacyReportPrinter();
    GradeExporter adapter = new GradeExporterAdapter(legacy);
    ReportService service = new ReportService(adapter);
    service.generateReport("S1001", "Math: A, Science: B+");
}
}

```

//use case 2

//External SMS Gateway

// Target Interface

```

public interface NotificationSender {
    void send(String recipient, String message);
}

```

// Adaptee: external SMS library with incompatible interface

```

public class ThirdPartySMSTGateway {

```

```
public void dispatch(String phoneNumber, String body) {  
    System.out.println("[SMS Gateway] Sending to " + phoneNumber + ": " + body);  
}  
}
```

// Adapter

```
public class SMSGatewayAdapter implements NotificationSender {  
  
    private ThirdPartySMSGateway gateway;  
  
    public SMSGatewayAdapter(ThirdPartySMSGateway gateway) {  
        this.gateway = gateway;  
    }  
  
    @Override  
    public void send(String recipient, String message) {  
        gateway.dispatch(recipient, message);  
    }  
}
```

// Client

```
public class NotificationService {  
  
    private NotificationSender sender;  
  
    public NotificationService(NotificationSender sender) {
```

```
        this.sender = sender;
    }

    public void sendAttendanceAlert(String phone, String studentName) {
        sender.send(phone, "Attendance alert for student: " + studentName);
    }

    public static void main(String[] args) {
        ThirdPartySMSTGateway gateway = new ThirdPartySMSTGateway();
        NotificationSender adapter = new SMSTGatewayAdapter(gateway);
        NotificationService service = new NotificationService(adapter);
        service.sendAttendanceAlert("+201001234567", "Ahmed Kamal");
    }
}
```